

DATA301/DATA471 — Assignment IV

Luca Panettieri

24 September 2026

Contents

1	Exercise 1 — Estimators for the Exponential rate λ	1
1.1	Which definition of the sample quantiles? [10%]	2
1.2	A bootstrap bias-corrected IQR estimator [20%]	2
1.3	Monte Carlo comparison [20%]	3
2	Exercise 2 — Time-series analysis of ausbeer [50%]	6
2.1	Frequency of the series	6
2.2	Overall behaviour, trend, seasonality, variance, outliers	6
2.3	Seasonal plot	8
2.4	Seasonal subseries plot	8
2.5	Would I detrend or transform before modelling?	8
3	References	9

1 Exercise 1 — Estimators for the Exponential rate λ

I model the time between arrivals as i.i.d. $X_1, \dots, X_n \sim \text{Exponential}(\lambda)$, with $\lambda > 0$ the reciprocal of the mean. I compare three estimators of λ : the maximum likelihood estimator, an analogy estimator built from the inter-quartile range (IQR), and a bootstrap bias-corrected version of that IQR estimator.

```
# Maximum likelihood: lambda_ML = 1 / mean(X) (= n / sum(X))
lambda_ml <- function(x) 1 / mean(x)

# Fast type-7 quantile (identical to quantile(x, p, type = 7)), used so the
# Monte Carlo below runs in reasonable time.
q7 <- function(xs, p) {
  # xs must already be sorted
  n <- length(xs); h <- 1 + (n - 1) * p; lo <- floor(h); fr <- h - lo
  if (lo >= n) xs[n] else xs[lo] + fr * (xs[lo + 1] - xs[lo])
}

# IQR / analogy estimator: lambda_IQR = ln(3) / (Q3 - Q1).
# For Exp(lambda), Q3 - Q1 = ln(3)/lambda, which is what makes this consistent.
```

```
lambda_iqr <- function(x) {
  xs <- sort(x)
  log(3) / (q7(xs, 0.75) - q7(xs, 0.25))
}
```

1.1 Which definition of the sample quantiles? [10%]

Both Wicklin (2017) and Hyndman & Fan (1996) — the latter also cited in the machine-learning lectures — make the same basic point: “the quartile” is not a single thing. A sample quantile has to interpolate between the ordered data points, and there are nine common ways of doing that. Hyndman & Fan collect them into one family indexed by where the p -th quantile sits, $h = (n - a)p + a + b$ for constants that change with the definition. The nine split into two groups: types 1–3 are *discontinuous* (they jump from one order statistic to the next, essentially reading off the empirical CDF), while types 4–9 are *continuous*, drawing a piecewise-linear line through the order statistics and interpolating along it. Wicklin’s blog walks through the same nine definitions from the SAS side and shows, with pictures, that they only really disagree in small samples and out in the tails; for large n they all converge to the same number.

Hyndman & Fan recommend **type 8**, because its quantile estimates are approximately median-unbiased whatever the underlying distribution is, so it doesn’t bake in an assumption about the data. The practical default is different: **R’s quantile() uses type 7**, and NumPy’s default (`method = "linear"`) is the same type 7. Type 7 is the piecewise-linear interpolation of the empirical CDF and is exactly median-unbiased only for the uniform.

For this assignment I use **type 7** — R’s default. My reason is that the data are continuous Exponential draws with (almost surely) no ties, so the gap between type 7 and type 8 is $O(1/n)$ and disappears well before it could change any of the conclusions in the Monte Carlo study below. Type 8 would be the better choice if I cared about a single, distribution-robust point estimate of a quartile; here I care about the *relative* behaviour of three estimators, and that is not sensitive to the quantile convention. I note the default explicitly so the choice is deliberate rather than accidental.

1.2 A bootstrap bias-corrected IQR estimator [20%]

The IQR estimator is a nonlinear function of two sample quantiles, so it is biased in finite samples. The bootstrap (Efron, 1979; Davison & Hinkley, 1997) gives an estimate of that bias without any analytic work, and it is the resampling idea introduced in the machine-learning lectures. Write $\hat{\theta} = \hat{\lambda}_{\text{IQR}}(X)$. Resample the data with replacement, recompute the estimator on each resample to get $\hat{\theta}_1^*, \dots, \hat{\theta}_B^*$, and estimate the bias by $\widehat{\text{bias}} = \overline{\hat{\theta}^*} - \hat{\theta}$. The first-order bias-corrected estimator subtracts that estimate:

$$\hat{\lambda}_{\text{BC}} = \hat{\theta} - \widehat{\text{bias}} = 2\hat{\theta} - \overline{\hat{\theta}^*}.$$

I fix $B = 200$ replications per input sample, as allowed.

```
lambda_iqr_bc <- function(x, B = 200) {
  theta_hat <- lambda_iqr(x)
  n <- length(x)
  theta_star <- numeric(B)
  for (b in seq_len(B)) {
    theta_star[b] <- lambda_iqr(x[sample.int(n, n, replace = TRUE)])
  }
  2 * theta_hat - mean(theta_star) # = theta_hat - (mean(theta_star) - theta_hat)
}
```

```
x_demo <- rexp(50, rate = 1) # quick check on one sample (not the experiment)
c(ML = lambda_ml(x_demo), IQR = lambda_iqr(x_demo), IQR_BC = lambda_iqr_bc(x_demo))
```

```
##          ML          IQR      IQR_BC
## 0.9461495 0.9852692 0.9241314
```

1.3 Monte Carlo comparison [20%]

I follow the structure set out in the Monte Carlo lecture (Frery, 2026). The metric there is the mean squared error, which decomposes as

$$\text{MSE}(\hat{\lambda}) = B^2(\hat{\lambda}) + \text{Var}(\hat{\lambda}), \quad B(\hat{\lambda}) = E(\hat{\lambda}) - \lambda,$$

so bias and variance are the two things that make up the score. The lecture’s “first mandatory approach” is to compute the MSE analytically; only when that fails do we fall back on simulation. Here the split is instructive: the ML estimator is analytically tractable ($n/\sum X_i$ is an inverse-gamma variable with known moments), but the IQR estimator — and especially its bootstrap bias-corrected version, which has a random bootstrap step inside it — has no closed-form MSE. That intractability is exactly what justifies a Monte Carlo experiment.

Following the lecture’s algorithm, each combination of λ and n is a *situation*; I sweep $\lambda \in \{0.1, 1, 10\}$ and $n \in \{5, 30, 100\}$ (so $3 \times 3 = 9$ situations), run R replications per situation, and in each replication draw a fresh sample from $\text{Exponential}(\lambda)$ and store all three estimates. The MSE estimate for a situation is $\widehat{\text{MSE}} = \frac{1}{R} \sum_{r=1}^R (\hat{\lambda}_r - \lambda)^2$; I also record its Monte Carlo standard error, the estimated bias and variance (to read off the decomposition above), and the fraction of replications that produced a *finite* estimate — the last matters at $n = 5$, as we will see.

```
lambdas <- c(0.1, 1, 10)
ns      <- c(5, 30, 100)
R_MC    <- 1000 # replications per cell; increase for tighter SEs (slower)

run_cell <- function(lambda, n, R = R_MC) {
  est <- matrix(NA_real_, R, 3, dimnames = list(NULL, c("ML", "IQR", "IQR_BC")))
  for (r in seq_len(R)) {
    x <- rexp(n, rate = lambda)
    est[r, 1] <- lambda_ml(x)
    est[r, 2] <- lambda_iqr(x)
    est[r, 3] <- lambda_iqr_bc(x)
  }
  sq <- (est - lambda)^2
  mse <- colMeans(sq) # Inf if any estimate is Inf
  se <- apply(sq, 2, sd) / sqrt(R)
  pfin <- colMeans(is.finite(est)) # share of finite estimates
  # bias and variance from the FINITE estimates, for the MSE = bias^2 + var view
  bias <- apply(est, 2, function(v) mean(v[is.finite(v)]) - lambda)
  vare <- apply(est, 2, function(v) var(v[is.finite(v)]))
  data.frame(lambda, n, estimator = names(mse),
             mse = as.numeric(mse), mc_se = as.numeric(se),
             bias = as.numeric(bias), variance = as.numeric(vare),
             p_finite = as.numeric(pfin), row.names = NULL)
}

grid <- expand.grid(lambda = lambdas, n = ns)
results <- do.call(rbind, Map(run_cell, grid$lambda, grid$n))
```

```
mse_wide <- reshape(results[, c("lambda", "n", "estimator", "mse")],
  idvar = c("lambda", "n"), timevar = "estimator",
  direction = "wide")
names(mse_wide) <- sub("^mse\\.\\.", "", names(mse_wide))
knitr::kable(mse_wide, digits = 4,
  caption = "Estimated MSE by estimator for each (lambda, n).")
```

Table 1: Estimated MSE by estimator for each (lambda, n).

	lambda	n	ML	IQR	IQR_BC
1	0.1	5	0.0044	1.1658	Inf
4	1.0	5	0.7320	35.1329	Inf
7	10.0	5	67.7532	1458.9554	Inf
10	0.1	30	0.0004	0.0011	0.0013
13	1.0	30	0.0407	0.1374	0.1449
16	10.0	30	3.7935	9.9395	10.8241
19	0.1	100	0.0001	0.0003	0.0003
22	1.0	100	0.0101	0.0250	0.0293
25	10.0	100	0.9964	2.4659	2.7960

```
# Fraction of finite IQR-BC estimates -- shows where the estimator breaks down.
bc <- results[results$estimator == "IQR_BC", c("lambda", "n", "p_finite")]
bc_wide <- reshape(bc, idvar = "lambda", timevar = "n", direction = "wide")
names(bc_wide) <- sub("^p_finite\\.\\.", "n=", names(bc_wide))
knitr::kable(bc_wide, digits = 3,
  caption = "Fraction of finite IQR-BC estimates (by lambda x n).")
```

Table 2: Fraction of finite IQR-BC estimates (by lambda x n).

	lambda	n=5	n=30	n=100
3	0.1	0	1	1
6	1.0	0	1	1
9	10.0	0	1	1

```
# Bias^2 + variance decomposition for IQR vs IQR-BC, at lambda = 1 (finite cells).
# Shows WHY the bias correction loses on MSE: it trims bias but adds variance.
dec <- results[results$lambda == 1 & results$n %in% c(30, 100) &
  results$estimator %in% c("IQR", "IQR_BC"),
  c("n", "estimator", "bias", "variance", "mse")]
dec$bias_sq <- dec$bias^2
knitr::kable(dec[, c("n", "estimator", "bias", "bias_sq", "variance", "mse")],
  digits = 4, row.names = FALSE,
  caption = "Bias / variance decomposition at lambda = 1 (bias-squared + variance = MSE).")
```

Table 3: Bias / variance decomposition at $\lambda = 1$ (bias-squared + variance = MSE).

n	estimator	bias	bias_sq	variance	mse
30	IQR	0.1427	0.0204	0.1171	0.1374
30	IQR_BC	0.0409	0.0017	0.1434	0.1449
100	IQR	0.0378	0.0014	0.0236	0.0250
100	IQR_BC	0.0149	0.0002	0.0291	0.0293

```
plot_df <- results[is.finite(results$mse), ]
ggplot(plot_df, aes(factor(n), mse, colour = estimator, group = estimator)) +
  geom_line() + geom_point() +
  facet_wrap(~ lambda, scales = "free_y", labeller = label_both) +
  scale_y_log10() +
  labs(x = "Sample size n", y = "MSE (log scale)", colour = "Estimator") +
  theme_minimal()
```

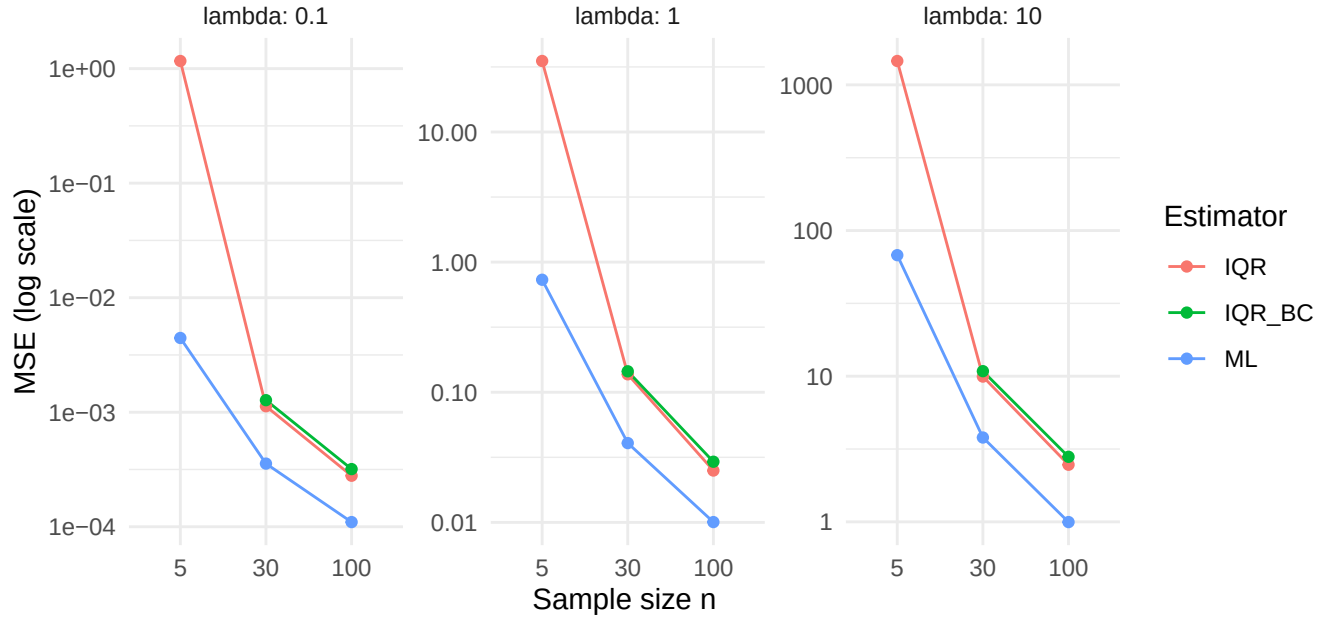


Figure 1: MSE against sample size, faceted by λ (log MSE). Non-finite points (IQR-BC at $n=5$) are omitted.

Number of replications. The lecture notes that a Monte Carlo estimate of a mean has imprecision proportional to Var/R , so its standard error falls like $1/\sqrt{R}$; that is the lever for choosing R . Here I use a common $R = 1000$ across situations rather than a situation-specific R_n , and check it after the fact: the `mc_se` column (computed but not printed to save space) is around 5–9% of the corresponding MSE for every $n = 30$ and $n = 100$ cell — much smaller than the gaps between estimators, which start at about $2.5\times$ and run far higher. So $R = 1000$ already separates the estimators cleanly at those sample sizes, and pushing R higher would only shrink error bars that are already well inside the differences I am interpreting. The one place R cannot help is $n = 5$: there the problem is not sampling noise but that the estimator is undefined (below), and no number of replications fixes that — the fix would be a different estimator, not a larger R .

What the results say.

- **ML wins everywhere.** It has the lowest MSE in every cell, which is what theory predicts: when the model is correct the MLE is asymptotically efficient, and here the model *is* the data-generating process.
- **IQR pays for using only two quantiles.** At $n = 30$ and $n = 100$ its MSE is roughly $2.5\text{--}3.4\times$ the ML MSE. That gap reflects its lower asymptotic efficiency — it throws away most of the sample and keeps only the quartile spacing.
- **The bias correction makes MSE worse, not better.** For $n = 30$ and $n = 100$ the bias-corrected estimator has MSE about 5–17% *above* plain IQR. The bias/variance table makes the reason concrete: reading the $\text{MSE} = B^2 + \text{Var}$ decomposition, the correction does shrink the bias term, but it adds more variance than the bias it removes, so the sum goes up. This is the bias–variance trade-off from the Monte Carlo lecture, and a useful reminder that a bias correction is not automatically an improvement on MSE.
- **At $n = 5$ the IQR estimators break down.** The plain IQR estimate is finite but wildly variable — its MSE runs from roughly $20\times$ up to $260\times$ the ML MSE across these three cells, and that multiple is itself unstable from run to run. The bias-corrected estimate is *infinite*: the finite-fraction table shows essentially none of the bootstrap-based estimates are usable at $n = 5$. The reason is structural — a bootstrap resample of only five distinct values very often repeats a value so that $Q_3 = Q_1$, the denominator $\ln 3/(Q_3 - Q_1)$ blows up, and a single infinite resample makes $\hat{\theta}^*$, and hence $\hat{\lambda}_{\text{BC}}$, infinite. The lesson is that bootstrapping quantile-spacing estimators needs a sample large enough to have several distinct order statistics; $n = 5$ is below that threshold.
- **Scaling checks.** MSE grows like λ^2 across the columns (a factor of ~ 100 per factor of 10 in λ), as expected for a scale parameter, and falls roughly like $1/n$ down the rows. Both are sanity checks that the experiment behaves sensibly.

Overall: if the exponential model is trusted, use ML. The IQR estimators are only worth considering for their robustness to model departures (not tested here), and if they are used the bootstrap bias correction should be avoided at small n and treated with caution elsewhere, since it costs MSE.

2 Exercise 2 — Time-series analysis of ausbeer [50%]

`ausbeer` records quarterly Australian beer production (megalitres). I analyse the recent, complete-looking span from 1992 onward; the choice is stated so the seasonal plots are read over a period with a stable definition of the series.

```
beer <- window(ausbeer, start = 1992)
```

2.1 Frequency of the series

```
frequency(beer)
```

```
## [1] 4
```

The frequency is **4**: the data are quarterly. I can confirm this two ways — the `ts` object reports a frequency of 4, and the time plot below shows a pattern that repeats every four observations (one cycle per year).

2.2 Overall behaviour, trend, seasonality, variance, outliers

```
autoplot(beer) + labs(x = "Year", y = "Megalitres") + theme_minimal()
```

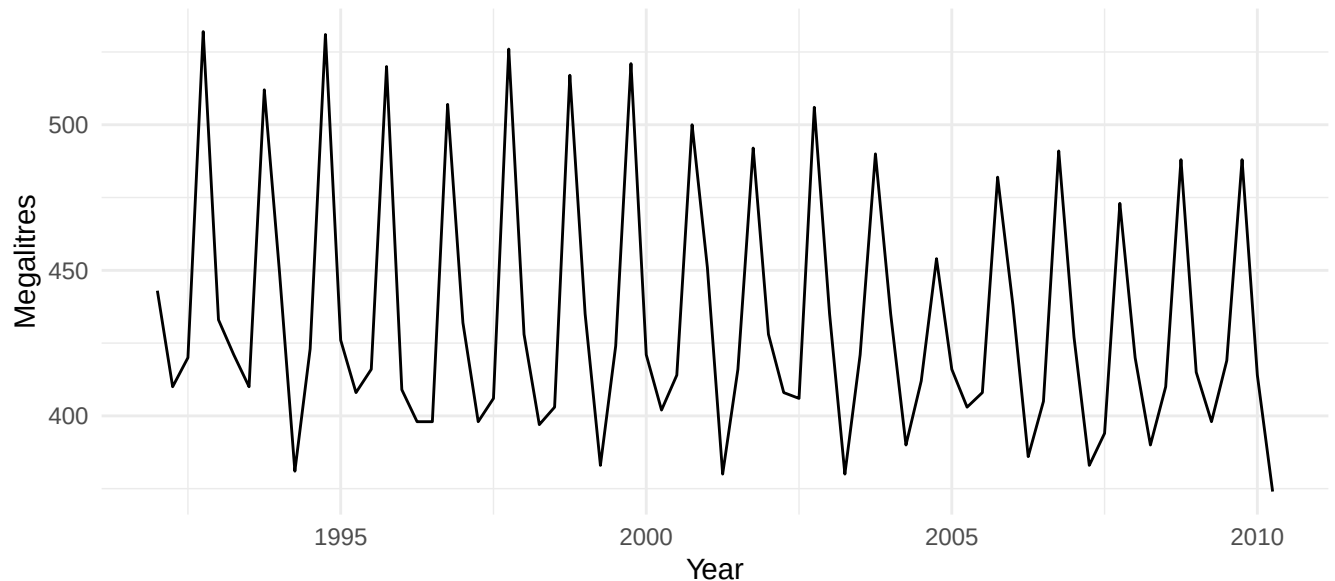


Figure 2: Quarterly Australian beer production, 1992 onward.

Trend (Q2). There is no strong long-term trend, but a slight downward drift: a straight line through the series falls by roughly 1.3 megalitres per year over 1992–2010. Production is broadly flat with a gentle decline, not a clear rise.

Seasonality (Q3). Yes, there is clear seasonality. In the lecture’s terms this is *periodic* behaviour — it repeats identically on a fixed four-quarter period — rather than the looser *cyclic* behaviour whose length wanders, so it is genuine seasonality tied to the calendar. The quarterly means are $Q1 \approx 429$, $Q2 \approx 394$, $Q3 \approx 411$, $Q4 \approx 502$: $Q4$ stands well above the rest and $Q2$ is the lowest. That fits the southern-hemisphere calendar — the $Q4$ spike is the run-up to the Christmas/summer period.

Variance (Q4). The size of the seasonal swing looks fairly constant in absolute terms across the whole window — the peaks and troughs don’t visibly fan out or shrink over time. The automatic Box-Cox estimate is a boundary value (below), but the visual evidence is that variability is roughly stable, so a variance-stabilising transformation is not clearly needed.

```
BoxCox.lambda(beer)      # Guerrero method; note it can sit on the search boundary
```

```
## [1] -0.9999242
```

The Guerrero estimate returns $\lambda \approx -1$, which is the lower end of its search range. I read that as the automatic method being pulled to the boundary rather than as strong evidence for an inverse transformation — visually the amplitude is stable, so I would not lean on it (see Q8).

Unusual observations (Q5). The most eye-catching point is the very last one, 2010 Q2, at 374 ML — the lowest value in the window. Some of that is simply that $Q2$ is always the seasonal low, but it also sits at the bottom of the recent decline, which makes it stand out. The early $Q4$ peaks (1992 $Q4$ is the maximum at 532 ML) are the other extreme. Nothing looks like a data error; the extremes line up with the seasonal pattern and the mild downward drift.

2.3 Seasonal plot

```
ggseasonplot(beer, year.labels = TRUE) + labs(y = "Megalitres") + theme_minimal()
```

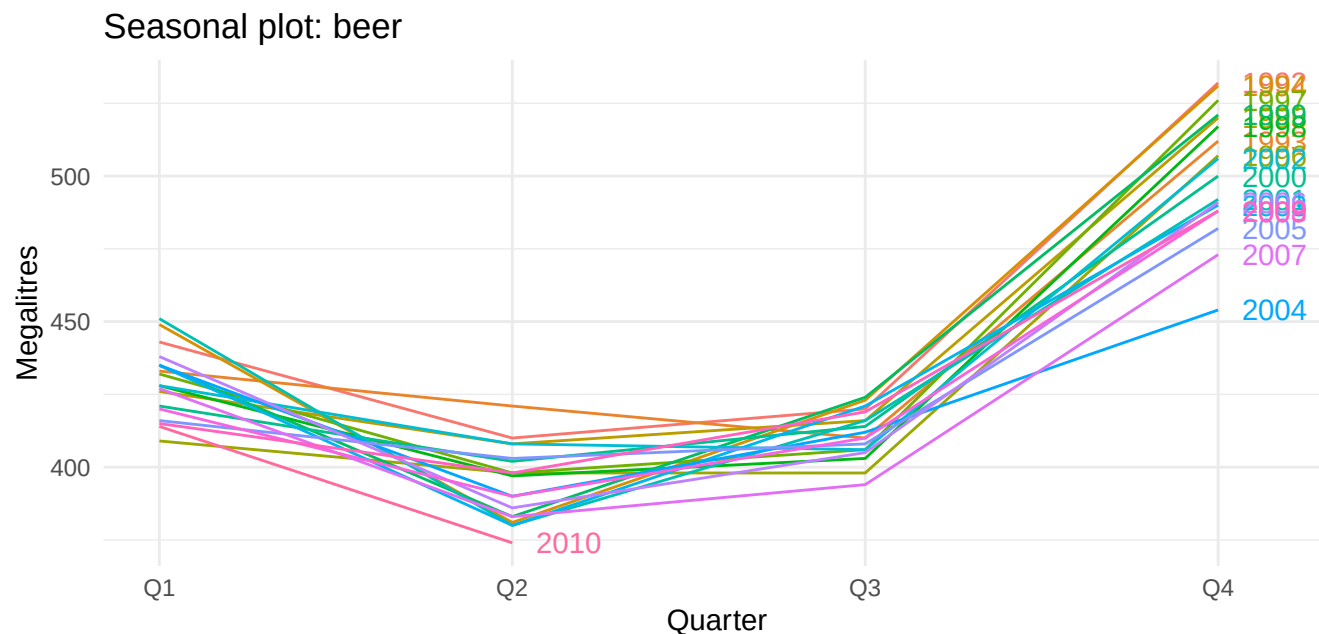


Figure 3: Seasonal plot: each year drawn as a line across the quarters.

Q6. Overlaying the years makes the shape of the seasonal pattern obvious in a way the ordinary plot does not: almost every year traces the same profile — up into Q4, down to a Q2 trough — and the lines sit roughly parallel. That parallelism is the signal that the seasonal *pattern* is stable and that the year-to-year differences are mostly a shift in level, not a change in shape.

2.4 Seasonal subseries plot

```
ggsubseriesplot(beer) + labs(y = "Megalitres") + theme_minimal()
```

Q7. Reading off the blue quarter means, **Q4 is clearly the highest** and **Q2 the lowest**, with Q1 and Q3 in between. Within each quarter's panel the values drift down slightly over time, mirroring the mild overall decline, but the *ranking* of the quarters never changes — so the seasonal pattern is stable across the whole period, even as the level edges down.

2.5 Would I detrend or transform before modelling?

Q8. The lecture frames this as a question about **stationarity**: classical time-series methods assume, or work best on, a weakly stationary series — one whose mean, variance and autocorrelation stay constant over time — and the point of transforming or differencing is to get closer to that. Reading the plots against that standard:

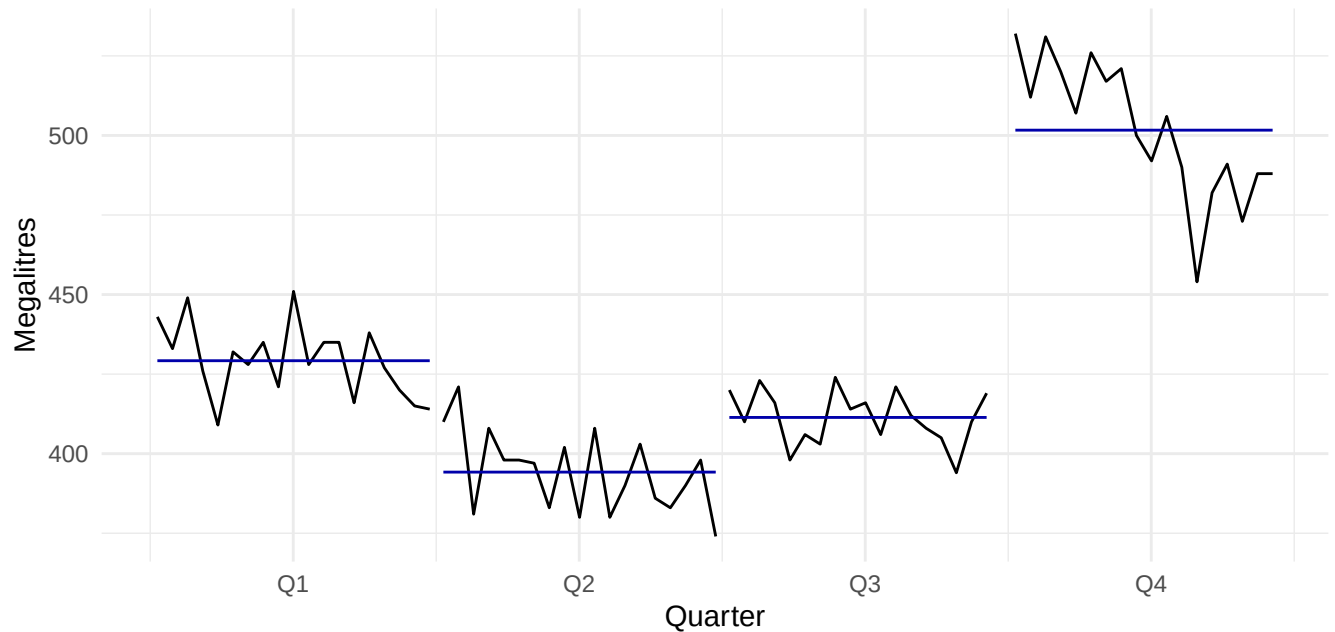


Figure 4: Seasonal subseries plot: each quarter shown as its own mini-series over time.

- *Variance.* The seasonal amplitude is stable across the window, so the variance condition is roughly met already. I would **not** apply a variance-stabilising transformation, despite the automatic Box-Cox value sitting at its boundary — that boundary result isn't backed by any fanning in the picture.
- *Mean.* The only thing breaking stationarity is the seasonality plus a weak downward drift, both of which move the mean. Rather than log-transforming or hard-detrending up front, the natural route to stationarity here is a **seasonal difference** (lag 4), which would remove the periodic pattern and the mild drift together and leave something close to white noise.

So my answer is: no transformation, and no separate detrending — if anything were needed before modelling it would be a single seasonal difference to reach stationarity. (As instructed, I fit no model here; this is a visual argument only.)

3 References

- Davison, A. C., & Hinkley, D. V. (1997). *Bootstrap Methods and their Application*. Cambridge University Press.
- Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1), 1–26.
- Frery, A. C. (2026). *The Monte Carlo Method* [DATA301 lecture]. Victoria University of Wellington.
- Hyndman, R. J., & Athanasopoulos, G. (2018). *Forecasting: Principles and Practice*. OTexts. <https://OTexts.com/fpp2>
- Hyndman, R. J., & Fan, Y. (1996). Sample quantiles in statistical packages. *The American Statistician*, 50(4), 361–365.
- Robert, C. P., & Casella, G. (2009). *Introducing Monte Carlo Methods with R*. Springer.
- Wicklin, R. (2017). Sample quantiles: A comparison of 9 definitions. *SAS Blogs*. <https://blogs.sas.com/content/iml/2017/05/24/definitions-sample-quantiles.html>